ROS 2 & Gazebo Simulation

Engineering Validation and Demonstration Report

Swarm-SAR | ubuntu_simulations

Ubuntu 26.04.1 | ROS 2 Lyrical | Gazebo Sim 10.5.0

Validation date: 28 October 2026

Evidence status: 0 original PNG screenshots supplied; target is 10.

This report separates observed results from implementation claims. It is not a competition-readiness certificate. Missing evidence and failed tests are retained explicitly.

Executive summary

The existing four-package architecture was retained. Configurable robot scenarios, optional GUIs, reduced sensor workloads and filtered bridges make smaller demonstrations possible. Runtime testing found an OpenGL failure, missing IMU publication and double-applied drone spawn offsets; the latter two were corrected and retested.

The first run failed in Ogre rendering. A software-rendered pair ran with real scans, camera images, odometry and peer traffic. The first software test lacked IMU data and showed incorrect drone altitude. See the corrected observations below for the final measured state.

Architecture and engineering changes

Gazebo physics and sensor plugins -> ROS/Gazebo bridge -> each robot's decentralized agent. Agents publish velocity commands through the bridge back to Gazebo. The RF transport delivers peer state with range, loss and delay. The survivor oracle emits synthetic, noisy evidence. It is a simulation proxy, not a real thermal vision detector.

Namespaces remain rover_0, rover_1, drone_0 and drone_1 to preserve compatibility. The pair scenario starts rover_0 and drone_0 simultaneously. Rover odometry is spawn-relative; drone odometry is world-relative. Map transforms and agent/radio/oracle offsets now reflect that distinction.

Drone TF follows map -> drone_0/odom -> drone_0/base_footprint -> drone_0/base_link -> sensor_link. Three-dimensional drone odometry enables altitude feedback. The IMU world system now updates IMU sensors.

Low-resource mode uses 180 LiDAR rays at 5 Hz and a 320 x 240 camera at 3 Hz; full sensor settings are selectable. RViz is disabled by default. Headless mode retains rendering for GPU LiDAR/camera sensors, so it still needs a usable EGL/OpenGL implementation.

Validation results

Build: all four packages passed. Existing checks: five static integration tests and three helper tests passed. Xacro expansion confirmed 720 rays/10 Hz versus 180 rays/5 Hz for the rover.

Initial headless launch: FAIL (OpenGL context failure). First software pair: PARTIAL (sensors and movement observed, IMU missing, drone offset bug). Shutdown of that bounded run required SIGKILL after its grace period: FAIL for clean shutdown.

Corrected GUI shutdown: all ROS nodes exited cleanly after terminal Ctrl+C. Gazebo exited with signal code -2 without forced kill. This is successful interruption, but Gazebo still reports a nonzero exit.

Corrected live pair: missing requested streams = []

/clock: 4024 messages in 35.1 wall seconds.

/rover_0/imu/data: 810 messages in 35.1 wall seconds.

/drone_0/imu/data: 1614 messages in 35.1 wall seconds.

/tf: 899 messages in 35.1 wall seconds.

/rover_0/odom: 450 messages in 35.1 wall seconds.

/drone_0/odom: 450 messages in 35.1 wall seconds.

/drone_0/downward_camera/image_raw: 49 messages in 35.1 wall seconds.

/rover_0/scan: 81 messages in 35.1 wall seconds.

/drone_0/scan: 82 messages in 35.1 wall seconds.

/drone_0/status: 16 messages in 35.1 wall seconds.

/rover_0/status: 16 messages in 35.1 wall seconds.

/rover_0/peer_inbox: 14 messages in 35.1 wall seconds.

/drone_0/peer_inbox: 13 messages in 35.1 wall seconds.

/rover_0/odom: first [-0.0, -0.0, 0.0]; last [2.81, -3.532, 0.0].

/drone_0/odom: first [-16.0, -14.0, 2.5]; last [-16.676, -15.3, 2.451].

Reproducibility

```
cd /home/yash/Desktop/swarm-sar/ros2_ws
source /opt/ros/lyrical/setup.bash
colcon build --symlink-install
source install/setup.bash
LIBGL_ALWAYS_SOFTWARE=1 ros2 launch swarm_sar_gazebo \
  swarm_sar_sim.launch.py scenario:=pair
```

For one rover use scenario:=rover; one drone: scenario:=drone; four robots: scenario:=full. Add headless:=true for server-only operation, rviz:=true to show RViz, and low_resource:=false for full sensor settings. Four-robot operation was not revalidated during this audit.

From the repository root in another sourced terminal: python3 scripts/ros2_runtime_probe.py --seconds 30. For a single robot pass --agents rover_0 or --agents drone_0. Peer traffic is checked only for multi-robot runs.

Stop with Ctrl+C in the launch terminal. Do not overlap launches. The raw failed and successful observations are preserved under evidence/2026-10-28/logs. Dependencies are declared in package.xml; resolve with rosdep before building on a fresh machine.

Limitations and future work

The drone uses a body velocity controller rather than rotor aerodynamics. Mapping uses odometry and approximates sensor offsets; the reactive obstacle barrier does not guarantee collision avoidance. Initial rover ground contact, wheel TF, long-duration coverage, survivor recovery and command-step response need further tests. Synthetic health/battery values are not hardware diagnostics.

The VM has 7.3 GiB RAM and no swap. GNOME denied direct screenshot capture; the user offered manual capture. Ten original screenshots remain a required deliverable until supplied. No screenshots, terminal displays or simulation results have been fabricated.

Official graphics troubleshooting reference: https://gazebosim.org/docs/latest/troubleshooting/

Screenshot evidence register

01. World overview: awaiting manual capture and review

02. Robot model detail: awaiting manual capture and review

03. ROS runtime nodes: awaiting manual capture and review

04. Topics and sensor readings: awaiting manual capture and review

05. ROS/Gazebo bridge: awaiting manual capture and review

06. Robot movement: awaiting manual capture and review

07. Drone operation: awaiting manual capture and review

08. Simultaneous multi-robot scene: awaiting manual capture and review

09. TF and diagnostics: awaiting manual capture and review

10. Final demonstration scene: awaiting manual capture and review

Conclusion: source and configuration improvements are implemented and supported by bounded runtime observations. Complete acceptance requires the remaining genuine screenshots and unresolved validation gates.